

Knome Enterprise Knowledge Management Platform

Comprehensive Technical & Architectural Documentation

Project Name: Knome
Client / Enterprise: MPOnline Limited
Document Target: Developers, Technical Architects, System Administrators
Language: English
Version: 1.0.0

1. Executive Summary & Purpose

Knome is an enterprise-grade Knowledge Management and Social Collaboration Platform custom-designed for MPOnline Limited. It fuses capabilities inspired by LinkedIn (professional networking, connections, job portal), Medium (long-form articles, markdown publishing, rich text), YouTube (video streaming, media hosting, podcasts), and internal enterprise communications (departmental analytics, karma rewards, governance).

The architecture adheres strictly to a **Database-First Database Schema pattern** using **ASP.NET Core 10 Web API, Entity Framework Core, MS SQL Server**, and a modern **React (Vite + JavaScript/TypeScript)** single-page application frontend.

2. Technical Stack & Core Architectural Patterns

2.1 Technology Stack

Layer	Technologies & Tools
Backend API	ASP.NET Core 10, C# 13, Entity Framework Core 10, SQL Server
API Gateway	YARP (Yet Another Reverse Proxy), ASP.NET Core
BFF (Backend-For-Frontend)	Node.js, Next.js / Express Aggregation Layer
Frontend UI	React 18, Vite, JavaScript / TypeScript, Tailwind CSS, Lucide Icons
Real-time Services	ASP.NET Core SignalR Websockets (NotificationHub)
Security & Auth	JWT (JSON Web Tokens with HS256), BCrypt Password Hashing (Work Factor 11)
Background Processing	ASP.NET Core IHostedService (JobExpiryHostedService)
Validation & Mapping	FluentValidation, AutoMapper Profiles
Logging & Diagnostics	Serilog (Structured File & Console Logging)

2.2 Architectural Principles & Constraints

- Database-First Schema Source of Truth:** Database schema modifications originate exclusively from MS SQL Server. Scaffolded Entity Framework models (Models/) and DbContext (Data/KnomeDbContext.cs) are never modified manually.
- Repository & Service Pattern:**

- **Controllers:** Thin HTTP endpoints handling request mapping, response serialization, and status codes.
 - **Services:** Encapsulate all domain business rules, validation logic, permission checks, and transaction coordination.
 - **Repositories:** Pure data-access layer interacting directly with EF Core and SQL Server queries.
- 3. Response Envelope (ApiResponse):** Every API endpoint returns a standardized JSON structure containing Success boolean, Data payload, Message, and Errors collection.
- 4. Role-Based Access Control (RBAC):** Supports 4 primary roles:
- **Employee:** Content posting, browsing, commenting, connections, media consumption.
 - **Community Admin:** Managing specific communities, approving members, moderating posts.
 - **HR Administrator:** Posting jobs, broadcast messages, managing department structures.
 - **System Administrator:** Platform governance, audit log inspection, user suspension/restoration.
-

3. Backend Architecture (`Backend/Knome.API`)

The backend is built as a modular ASP.NET Core 10 Web API project organized into clean layer folders.

3.1 Controllers (`Backend/Knome.API/Controllers`)

Controllers handle HTTP requests and invoke corresponding application services.

- **KnomeControllerBase.cs:** Abstract base controller providing utility helper methods (`GetUserId()`, `GetUserRole()`, `standardized OkResponse()`, `ErrorResponse()`).
- **AuthController.cs:** Authentication endpoints for employee login, token issuance, and password verification.
- **UserController.cs:** User profile management, connection requests, followers/following, skills, interests, and user settings.
- **PostController.cs:** Social feed post CRUD operations, media uploads, attachments, and post filtering.
- **ArticleController.cs:** Markdown/rich text article publishing, draft saving, versioning, tag assignment, and category categorization.
- **VideoController.cs:** Video upload, streaming metadata management, video tagging, and view counter incrementing.
- **PodcastController.cs:** Podcast audio episode publishing, podcast series management, and audio playback tracking.
- **CommunityController.cs:** Community creation, member join/leave approval workflows, community feeds, and community administrative settings.
- **InteractionController.cs:** Social interactions including reactions (likes/claps), multi-level comments, bookmarks, and sharing options.
- **FeedController.cs:** Aggregated main feed generation, algorithmically combining posts, articles, videos, and podcasts based on user subscriptions.
- **SearchController.cs:** Global multi-entity search across users, posts, articles, videos, podcasts, and communities with search history tracking.
- **KarmaController.cs:** Gamification system tracking user Karma points, transaction histories, rewards, and leaderboards.
- **AnalyticsController.cs:** HR analytics and department engagement metrics dashboard reporting.
- **JobsController.cs:** Internal job portal posting, application submissions, and job status management.
- **NotificationsController.cs:** In-app notifications retrieval, mark-as-read status, and notification preferences.
- **AuditLogController.cs:** Administrative audit trail inspection recording platform activities and security events.
- **MediaController.cs:** Direct file upload handling for images, videos, audio clips, and documents.

3.2 Application Services (`Backend/Knome.API/Services`)

Services contain all core business logic and workflow orchestration.

- **AuthService.cs:** Handles BCrypt password validation, JWT token creation with claim claims (UserId, EmployeeId, Role, Email), and credential validation.

- `UserService.cs`: Manages employee profiles, skill badges, interests, department transfers, connection requests, and user suspensions.
- `PostService.cs`: Orchestrates post creation, attachment linking, content moderation checks, and feed notifications.
- `ArticleService.cs`: Manages article drafting, publishing workflow, tag generation, reading time estimation, and version history.
- `VideoService.cs`: Handles video processing metadata, playback duration parsing, and view metrics tracking.
- `PodcastService.cs`: Manages podcast audio episodes, series indexing, and global audio state coordination.
- `CommunityService.cs`: Enforces community privacy rules (Public vs Private), membership role permissions, and community post associations.
- `ContentInteractionService.cs`: Handles reactions, comments, bookmarking into custom categories, and share counts while triggering Karma point distribution.
- `FeedService.cs`: Implements feed relevance algorithms combining chronological order, popular posts, and user interest weights.
- `SearchService.cs`: Executes full-text SQL search queries across multiple entities and caches frequent search terms.
- `KarmaService.cs`: Business rules engine for awarding Karma points upon user engagements (creating posts, getting likes, writing top articles).
- `NotificationService.cs`: Formats and persists in-app notifications while pushing live updates via SignalR websockets.
- `JobService.cs`: Manages internal job vacancies, application workflows, and automated job expiration logic.
- `AuditLogService.cs`: Records immutable security and administrative action logs in SQL Server.
- `LocalFileStorageService.cs`: Local disk file storage driver for managing uploaded media files under `wwwroot/uploads`.
- `SuspensionGuard.cs`: Guard class verifying user account active/suspended state before processing sensitive operations.

3.3 Repositories (Backend/Knome.API/Repositories)

Repositories interact with Entity Framework Core to execute SQL queries.

- `Repository.cs`: Generic repository implementation for standard CRUD operations (`GetByIdAsync`, `GetAllAsync`, `AddAsync`, `UpdateAsync`, `DeleteAsync`).
- `UserRepository.cs`: Custom queries for user search, connection graph traversal, follower queries, and user skill/interest joins.
- `PostRepository.cs`: Specialized EF Core queries fetching posts with attachments, comment counts, author profiles, and community scopes.
- `ArticleRepository.cs`: Specialized queries for article tags, version histories, and reading metrics.
- `VideoRepository.cs`: Video retrieval queries with tag filters and streaming metadata.
- `PodcastRepository.cs`: Queries for podcast series groupings and episode listings.
- `CommunityRepository.cs`: Complex joins checking community membership status, admin roles, and private community access rights.
- `ContentInteractionRepository.cs`: Queries managing reactions, nested comments, custom bookmark categories, and shares.
- `FeedRepository.cs`: Multi-entity raw SQL / EF queries joining posts, articles, and media into a unified feed stream.
- `SearchRepository.cs`: SQL LIKE and full-text index queries executing across users, posts, articles, videos, and podcasts.
- `KarmaRepository.cs`: Karma balance updates, transaction logging, and top user leaderboard aggregation.
- `NotificationRepository.cs`: Queries for user notification queues and unread counter badges.
- `JobRepository.cs`: Job posting queries, applicant joins, and expired job state queries.
- `AuditLogRepository.cs`: Audit log retrieval queries with date range and user filtering.

3.4 Database Context & Models (Backend/Knome.API/Data & Models)

- `Data/KnomeDbContext.cs`: EF Core `DbContext` representing the MS SQL Server Knome database, containing entity sets and relational configurations.
- **Core Models** (`Backend/Knome.API/Models`):

- `User.cs`, `UserCredential.cs`, `Role.cs`, `Department.cs`: Authentication and organizational hierarchy entities.
- `Post.cs`, `PostAttachment.cs`: Feed post entities and file attachments.
- `Article.cs`, `ArticleAttachment.cs`, `ArticleTag.cs`, `ArticleVersion.cs`: Long-form publishing entities.
- `Video.cs`, `VideoTag.cs`: Video media entities.
- `Podcast.cs`, `PodcastSeries.cs`: Audio media entities.
- `Community.cs`, `CommunityMember.cs`, `CommunityPost.cs`: Community space entities.
- `Comment.cs`, `Reaction.cs`, `Bookmark.cs`, `Share.cs`: User interaction entities.
- `KarmaBalance.cs`, `KarmaTransaction.cs`: Gamification entities.
- `Job.cs`: Internal career opportunity entities.
- `Notification.cs`, `NotificationPreference.cs`: User notification entities.
- `AuditLog.cs`, `ModerationReport.cs`: Governance entities.
- `ConnectionRequest.cs`, `Follower.cs`, `UserSkill.cs`, `UserInterest.cs`: Networking entities.

3.5 Infrastructure Components

- `Background/JobExpiryHostedService.cs`: Background worker running periodically to check job posting expiration dates and set `IsActive = false`.
 - `Hubs/NotificationHub.cs`: SignalR hub enabling real-time websocket connections to deliver instant push notifications to active users.
 - `Extensions/ServiceCollectionExtensions.cs`: Extension methods configuring DI container bindings (Services, Repositories, DbContext, JWT Auth, Swagger).
 - `Extensions/ApplicationBuilderExtensions.cs`: Configures middleware pipeline (CORS, Static Files, Routing, Auth, Custom Exception Middleware).
 - `Middleware/ExceptionHandlingMiddleware.cs`: Intercepts unhandled runtime exceptions, logs details via Serilog, and outputs standard `ApiResponse` error payloads.
 - `Middleware/SecurityHeadersMiddleware.cs`: Appends HTTP security headers (X-Content-Type-Options, X-Frame-Options, Content-Security-Policy).
-

4. Frontend Architecture (`knomeUI/frontend`)

The frontend is a single-page web application developed with React, Vite, JavaScript/JSX, Tailwind CSS, and Lucide React icons.

4.1 Pages (`knomeUI/frontend/src/pages`)

- `Login.jsx`: User authentication view providing credential input, token storage in `localStorage`, and role-based redirect.
- `Dashboard.jsx`: Main user landing dashboard displaying personalized activity feeds, quick stats, karma score, and recommended connections.
- `Posts.jsx`: Dedicated feed view displaying short social posts, image galleries, and quick post creation input.
- `Articles.jsx`: Article hub showcasing featured long-form content, category filters, reading time indicators, and bookmark buttons.
- `ArticleView.jsx`: Rich article reader page displaying full markdown content, author details, tag list, claps/comments, and sharing modal.
- `Videos.jsx`: Video streaming gallery grid with category tabs, play previews, and upload triggers.
- `Podcasts.jsx`: Podcast audio platform featuring episode series, episode listings, and continuous background audio player integration.
- `Communities.jsx`: Directory listing public and private employee communities, join request triggers, and community discovery tabs.
- `CommunityView.jsx`: Detailed single community page displaying community banner, member roster, dedicated community feed, and admin controls.

- `Network.jsx`: Employee networking portal managing incoming connection requests, suggestions ("People You May Know"), and follower counts.
- `KarmaHistory.jsx`: Gamification page displaying total earned Karma points, detailed transaction log, and monthly leaderboard.
- `HRAnalytics.jsx`: HR dashboard visualizing department participation rates, top content creators, content consumption graphs, and job metrics.
- `Jobs.jsx`: Internal career portal displaying job openings, department filters, requirement details, and application modal triggers.
- `Profile.jsx`: Comprehensive user profile view showing bio, skills badges, interests, published posts/articles/videos, and connection status.
- `AdminConsole.jsx`: System administrator console for audit log review, content moderation reports, user account suspensions, and platform settings.
- `Search.jsx`: Global search page displaying tabbed search results across Users, Posts, Articles, Videos, Podcasts, and Communities.
- `SavedContent.jsx`: Bookmarked items portal allowing users to organize saved articles and posts into custom categories.

4.2 Component Architecture

Layout Components (`src/components/layout`)

- `Navbar.jsx`: Sticky top navigation bar containing global search input, navigation links, notification bell dropdown, audio player shortcut, and user profile avatar.
- `Sidebar.jsx`: Navigation sidebar offering quick access to feeds, media hubs, communities, HR analytics, and admin tools.
- `Footer.jsx`: Standard application footer with copyright and quick documentation links.
- `Layout.jsx`: Wrapper component combining Navbar, Sidebar, Footer, and main content area.
- `AuthGuard.jsx`: Protected route wrapper redirecting unauthenticated users to `/login`.
- `PageLoader.jsx`: Spinner overlay for lazy-loaded route transitions.

Modal Components (`src/components/modals`)

- `CreatePostModal.jsx`: Rich post creator supporting text, image/video attachment uploads, and community scope selection.
- `CreateArticleModal.jsx`: Comprehensive markdown article editor with cover image upload, tags, and category selection.
- `CreateCommunityModal.jsx`: Form modal for launching new communities with privacy settings and rules.
- `CreateJobModal.jsx`: HR vacancy creation modal for specifying job title, department, requirements, and closing date.
- `UploadVideoModal.jsx`: Video file uploader with title, description, thumbnail upload, and tag assignment.
- `UploadPodcastModal.jsx`: Audio podcast episode uploader with series selection and thumbnail graphics.
- `VideoPlayerModal.jsx`: Embedded video playback popup modal with view tracking.
- `ArticleShareModal.jsx`: Modal offering link copy, internal message sharing, and social sharing options.
- `SaveToCategoryModal.jsx`: Custom bookmark folder selection modal.
- `ReportModal.jsx`: Moderation report modal allowing users to flag inappropriate content.
- `NotificationSettingsModal.jsx`: Modal configuring email and push notification preferences.
- `ManagementModals.jsx` & `MediaUploadModals.jsx`: Reusable generic modal utilities.

Widget & Feed Components (`src/components/widgets`)

- `PostCard.jsx`: Universal content card rendering posts, author header, media preview, interaction buttons (like, comment, bookmark, share), and inline comment thread.
- `GlobalAudioPlayer.jsx`: Sticky bottom audio player supporting play/pause, seek bar, volume control, and continuous playback while navigating pages.
- `HotPostsWidget.jsx`: Trending content widget displaying top-performing posts of the week.
- `MyCommunitiesWidget.jsx`: Quick widget listing communities the logged-in user belongs to.
- `PeopleYouMayKnowWidget.jsx`: AI/algorithm recommendation widget suggesting colleague connections.
- `TrendingTagsWidget.jsx`: Widget displaying popular hashtag topics.

Context Providers (`src/components/context`)

- `UserContext.jsx`: React Context managing current user state, login/logout functions, and token persistence.
- `AudioContext.jsx`: Global Audio Context driving `GlobalAudioPlayer.jsx` state across page transitions.
- `ModalContext.jsx`: Centralized manager controlling open/close states of application modals.
- `ToastContext.jsx`: Global notification toast provider displaying success/error popup alerts.

API Services & Utilities (`src/utills`)

- `ApiClient.js`: Axios HTTP client configured with base URL (`http://localhost:5095/api`), default headers, and Request/Response interceptors appending Bearer JWT tokens.
 - `apiService.js`: Centralized JavaScript service wrapping all backend API endpoints for Auth, Users, Posts, Communities, Search, Notifications, and Analytics.
 - `articleService.js`: Frontend service dedicated to article API calls and draft caching.
 - `videoService.js`: Frontend service handling video API calls and stream formatting.
 - `karmaEngine.js`: Utility functions calculating karma levels and formatting point badges.
 - `engagementEngine.js`: Helper calculating engagement scores for feed ranking.
 - `restrictedWords.js`: Client-side content moderation word list for pre-validation.
-

5. Microservice Gateway & BFF Layers

5.1 API Gateway (`Gateway/Knome.Gateway`)

- `Program.cs`: ASP.NET Core entrypoint initializing YARP (Yet Another Reverse Proxy) services.
- `appsettings.json`: YARP configuration defining reverse proxy routes and cluster destinations, routing external traffic seamlessly to backend microservice instances.

5.2 BFF - Backend For Frontend (`Bff/knome-bff`)

- `src/`: Next.js / Express service serving as an aggregation proxy for web clients, consolidating multiple API requests into lightweight UI-tailored responses.
-

6. SQL Database & Utility Scripts

- `seed_posts.sql`: Database script seeding initial posts, comments, reactions, and test user balances.
 - `CreateConnectionRequests.sql`: SQL schema script initializing connection request tables and indexes.
 - `scratch_fix_users.sql`: Emergency repair script resetting user passwords to `Password@123` with valid BCrypt hashes.
 - `scratch_generate_pdf.py`: Python ReportLab generator converting Markdown documentation into styled PDF documents.
-

7. Build, Verification & Execution Instructions

7.1 Running the Backend API

```
cd Backend/Knome.API
dotnet restore
dotnet build -nologo
dotnet run          # API starts on http://localhost:5095 (Swagger at http://localhost:5095/swagger)
```

7.2 Running the Frontend UI

```
cd Frontend/knome-web # or knomeUI/frontend
npm install
npm run dev           # Vite server starts on http://localhost:5173
```

7.3 Verification Checkers

All verification tests run directly against live SQL Server (localhost, database Knome):

```
# Dependency Injection check across all 17 controllers and 16 services
dotnet run --project "Tools/VerifyDiResolvers/VerifyDiResolvers.csproj"

# Phase feature verification runner
dotnet run --project "Tools/VerifyPhase9/VerifyPhase9.csproj"
```

7.4 Test Login Credentials

- **User Accounts:** MP0101 (SYSADM), MP0102 (CADM), MP0103 (HRADM), MP0104–MP0106 (EMP)
- **Default Password:** Password@123

8. Role-Based User Interfaces & Access Specifications

Knome provides tailor-made User Interfaces for each of the 4 enterprise roles, enforcing strict client-side and server-side authorization:

8.1 System Administrator Interface (`SYSADM`)

- **Demo User:** MP0101 — Loveneesh Sharma
- **Primary UI Route:** /admin-console (AdminConsole.jsx)
- **Interface Features:**
 - **System Audit Logs Tab:** Live inspection of timestamped security events, IP addresses, user IDs, and API actions.
 - **Content Moderation Tab:** Review flagged posts, articles, videos, and podcasts with Approve or Dismiss options.
 - **Account Governance Tab:** Single-click User Account Suspension and Reactivation controls (SuspensionGuard.cs).
 - **Platform Preferences:** Manage global system settings, automated moderation thresholds, and restricted word lists.

8.2 HR Administrator Interface (`HRADM`)

- **Demo User:** MP0103 — Sourabh Sahu
- **Primary UI Route:** /hr-analytics (HRAnalytics.jsx) & /jobs (Jobs.jsx)
- **Interface Features:**
 - **HR Analytics Dashboard:** Visual department participation charts, content consumption graphs, and karma distribution metrics.
 - **Job Portal Management:** Launch new internal job vacancies using CreateJobModal.jsx, view applicant ratios, and edit/close active postings.
 - **Organization Broadcasts:** Post broadcast announcements (dashboardApi.getAnnouncements()) visible to all employees on the dashboard.

8.3 Community Administrator Interface (`CADM`)

- **Demo User:** MP0102 — Vishendra Sharma
- **Primary UI Route:** /community (Communities.jsx) & /community/view (CommunityView.jsx)
- **Interface Features:**
- **Community Creation & Settings:** Launch new communities (CreateCommunityModal.jsx) with public or private access control.
- **Join Request Management:** Review, accept, or decline pending member join applications for private communities.
- **Community Feed Moderation:** Manage community announcements, pins, and member role assignments.

8.4 Employee Interface (`EMP`)

- **Demo Users:** MP0104 (Rishikesh Ugle), MP0105 (Meghna Tiwari), MP0106 (Mayur Verma)
- **Primary UI Route:** / (Dashboard.jsx), /profile (Profile.jsx), /posts, /articles, /videos, /podcasts
- **Interface Features:**
- **Personalized Knowledge Feed:** Interactive dashboard feed, category filtering tabs (All, Articles, Videos, Podcasts, Jobs).
- **Media Publishing:** Short social posts, long-form markdown articles, video uploads, and podcast audio episodes.
- **Colleague Networking:** Connection requests, follow/unfollow, mutual colleague recommendations.
- **Gamification & Rewards:** Earn Karma points through engagements, view history (KarmaHistory.jsx), and track monthly leaderboards.

8.5 Interactive Role Switcher UI Banner (`RoleInterfaceBanner.jsx`)

The frontend includes a sticky interactive **Role Switcher Banner** rendered on the main dashboard (Dashboard.jsx), allowing developers, stakeholders, and testers to instantly toggle between SYSADM, HRADM, CADM, and EMP accounts with 1 click to experience each role's dedicated UI.

Documentation compiled and maintained for MPOnline Limited Knome Knowledge Management Platform.